efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

2024 / 2025

Les fondamentaux de l'algorithmique

(basé sur le cours d'Asma Gabis)

Rado Rakotonarivo
rado.rakotonarivo@efrei.fr

EESPIG
CONFÉRENCE DES GRANDES ÉCOLES
Cti

Plan du cours

- Qu'est-ce que l'algorithmique ?
- Les variables & types
- Les opérateurs
- Les entrées / sorties
- Les structures de contrôles : l'instruction conditionnelle
- Les structures de contrôles: les boucles
- Bonnes pratiques

Qu'est-ce que l'algorithmique ?

Qu'est-ce qu'un algorithme ?

Un algorithme est une séquence finie d'instructions non ambiguës permettant de résoudre un problème, ou de réaliser une tâche précise.

- Il permet de résoudre un problème donné
- Une solution qui peut facilement être traduite en un programme
- Pas besoin de maîtriser un langage de programmation
- Indépendant de tout langage de programmation

www.canstockphoto.fr

https://fr.dreamstime.com/

Chemin = Algorithme

Algorithmique

Qu'est-ce qu'un algorithme ?

- Les algorithmes sont utilisés dans la vie courante.
- Exemple: (https://www.youtube.com/watch?v=Fzh9S2nbBqo)
  - Problème : J'ai soif
  - Chemin vers la solution (algorithme)
    - Prendre un verre
    - Le mettre sous le robinet
    - Ouvrir le robinet
    - Attendre que le verre soit plein
    - Fermer le robinet
  - Solution: boire

https://www.educol.net/

Étapes de réussite d'un algorithme

1. Analyse

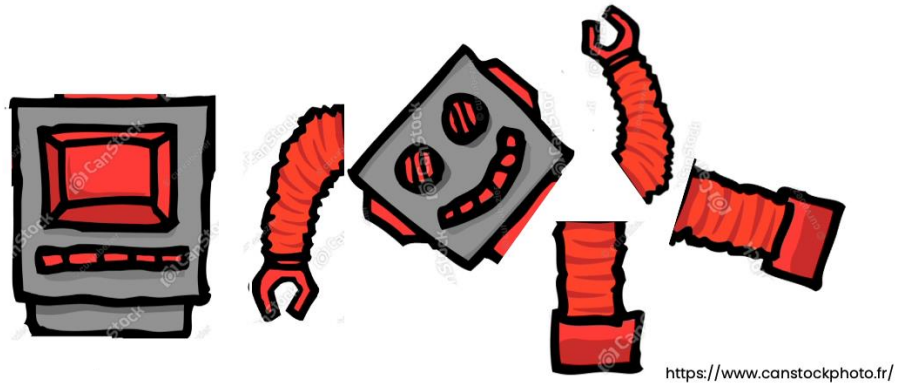
- Réflexion dans le but d'identifier le problème;
- Sélection des données à utiliser;
- Détermination claire du résultat attendu;
- Détermination des cas particuliers (s'il y en a);
- Le traitement à effectuer pour chaque situation;
- Découpage du problème en succession de tâches simples.

Étapes de réussite d'un algorithme

2. Conception
   - Transformation de données en résultats;
   - Détermination du séquencement des tâches élémentaires;
   - Prise en charge des cas particuliers;

Exemple 1: Assembler un robot

- Données: Parties d'un robot (1tête, 2 bras, 2 jambes , 1 corps)
- Tâches: assembler les parties
- Objectif: Obtenir un robot



Étapes de réussite d'un algorithme

2. Conception

Solution 1 (Algorithme 1) :

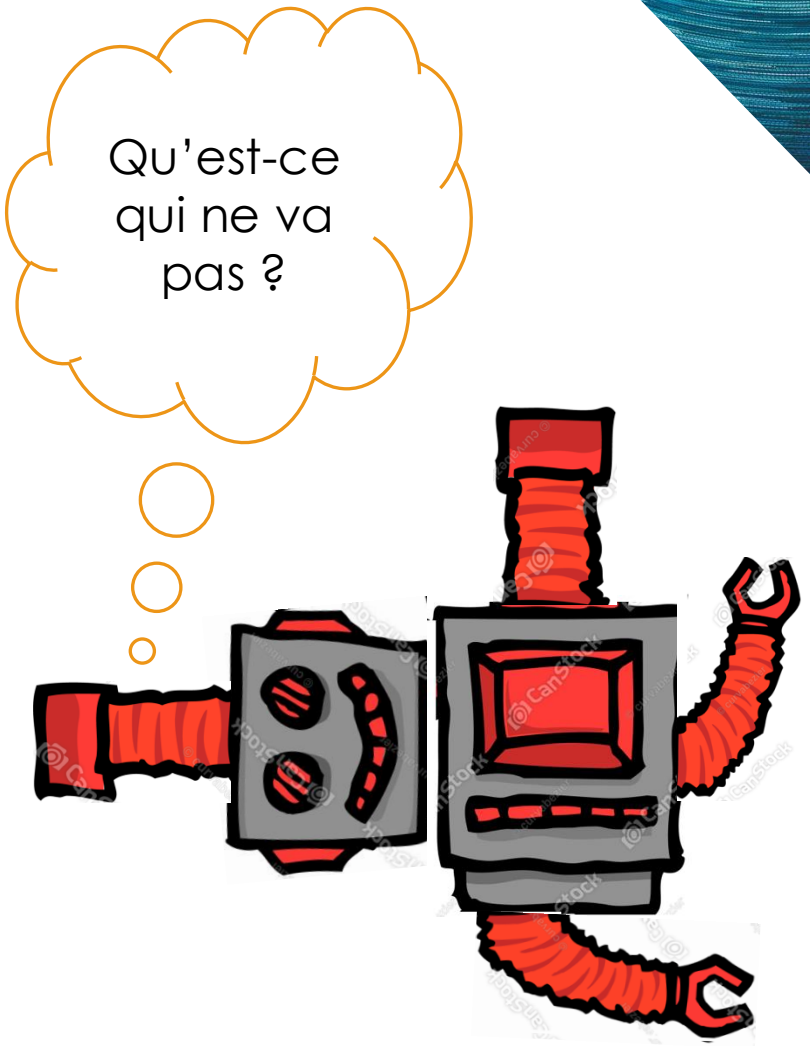
Placer la tête sur le corps

↓

Ajouter les deux bras

↓

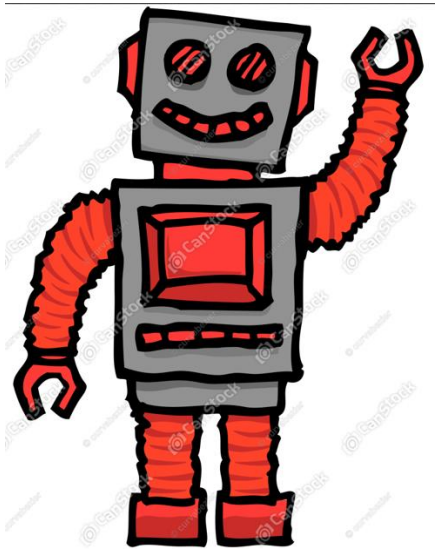
Ajouter les deux jambes



Étapes de réussite d'un algorithme

2. Conception

Solution 2 (Algorithme 2) :



- Placer la tête sur LE HAUT le corps
- Ajouter UN bras A GAUCHE du corps
- Ajouter UN bras A DROITE du corps
- Ajouter UNE jambe EN BAS A GAUCHE du corps
- Ajouter UNE jambe EN BAS A DROITE du corps

Étapes de réussite d'un algorithme

2. Conception

Règle N°1

Être précis et très clair dans la détermination de la séquence d'instructions

Étapes de réussite d'un algorithme

2. Conception

Exemple 2 : Se préparer pour sortir

Voici une suite d'étapes à suivre pour se préparer à sortir

Mettre jaquette

↓

Mettre bonnet

↓

Mettre chaussettes et chaussures

Étapes de réussite d'un algorithme

2. Conception

Règle N°2

Être précis sur l'ordre de la séquence d'instructions

Étapes de réussite d'un algorithme

3. Création de l'algorithme

Mettre en œuvre les actions élémentaires à l'aide d'une notation spécifique :

- Déclaration des variables
- Demande de données à l'utilisateur
- Boucles
- Conditions & test
- Affichage / Stockage du résultat
- ….

4. Encodage

Ecriture des actions en utilisant un langage de programmation: Instructions

Représentation d'un algorithme

3 façons de le faire :

Sous forme de texte : En décrivant les étapes à suivre.

Ingrédients : beurre, oeufs, sachets de sucre vanillé, farine, lait, sel

Récipients : saladier, verre mesureur, poêle, assiette

Opérations de base : ***mettre dans un récipient***, ***mélanger***, ***attendre pendant … minutes***, ***retourner***, ***laisser cuire pendant … minutes***

Algorithme des crêpes :

Mettre 4 oeufs **dans** le saladier
Mettre 1 sachet de sucre vanillé **dans** le saladier
Mettre 250 g de farine **dans** le verre mesureur
Mettre le contenu du verre mesureur **dans** le saladier
Mettre 0,5 litre de lait **dans** le verre mesureur
Mettre le contenu du verre mesureur **dans** le saladier
Mettre 50 grammes de beurre **dans** la poêle
Laisser cuire la poêle **pendant** 1 **minute**
Mettre le contenu de la poêle **dans** le saladier
Mélanger le contenu du saladier
Attendre pendant 60 **minutes**
Mettre 5 grammes de beurre **dans** la poêle

Laisser cuire la poêle **pendant** 0.5 **minute**
Tant que le saladier n'est pas vide :
- **Mettre** 5 cL du contenu du saladier **dans** le verre mesureur
- **Mettre** le contenu du verre mesureur **dans** la poêle
- **Laisser cuire** la poêle **pendant** 2 **minutes**
- **Retourner** le contenu de la poêle
- **Laisser cuire** la poêle **pendant** 2 **minutes**
- **Mettre** le contenu de la poêle **dans** l'assiette

Représentation d'un algorithme

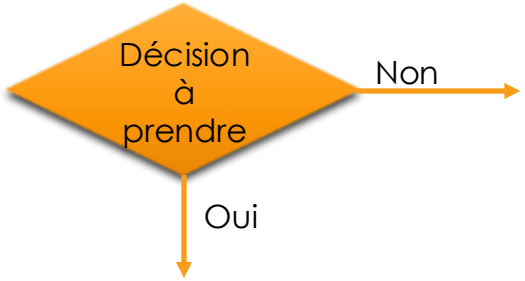
3 façons de le faire :

Sous forme d'un organigramme : Permet de décrire plus facilement qu'avec un texte le déroulement d'un cycle d'un système automatisé.

Cette représentation s'effectue avec 3 types de cases:

Un ovale correspond au début ou Fin (si fin il y a) de l'organigramme

Un rectangle correspond au une action à effectuer



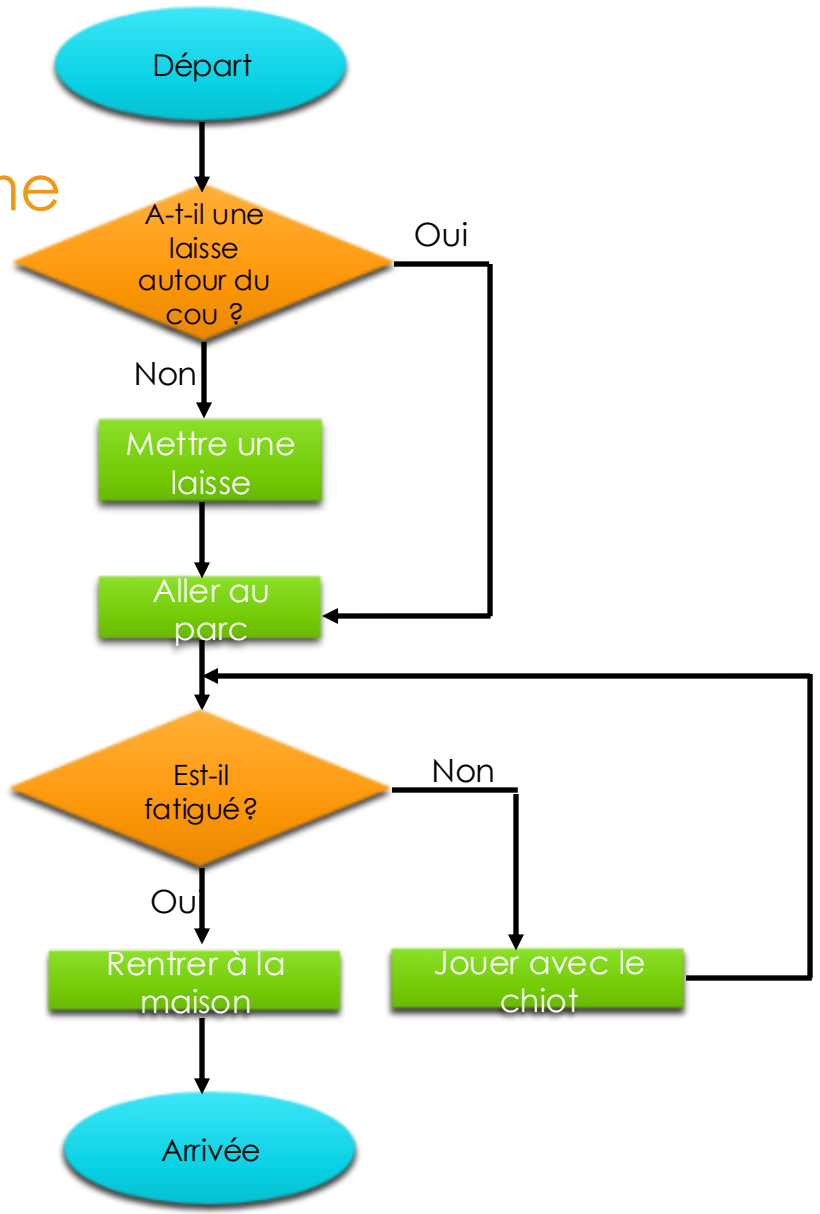
Un losange correspond à une question à laquelle on peut répondre uniquement par oui ou par non

Algorithmique

Représentation d'un algorithme

Sous forme d'un organigramme :

Exemple : Promener un chiot



Représentation d'un algorithme

3 façons de le faire :

Sous forme de pseudo-code: La forme qu'on utilisera le plus dans le cadre de ce cours.

```
Algorithme : Square2
Variables :
        X: entier (variable de type entier)
Début
    Ecrire(« Saisir une valeur entière: »)
    Lire(X)
    Ecrire(« Le carré de », X, « est », X*X)
Fin
```

Algorithmique

Quelle différence avec la programmation ?

- La programmation est la traduction d'un algorithme dans un langage compris par la machine appelé « langage de programmation » (Python, C, Java, …)
- Elle permet d'obtenir un résultat concret sur l'écran.

Pourquoi étudier l'algorithmique ?

- Savoir expliciter son raisonnement
- Savoir formaliser son raisonnement

Raisonnement valable et indépendant de tout langage de programmation

Que doit-on connaître ?

Pour écrire des algorithmes corrects :

1. Connaître la structure d'un algorithme
2. Connaître la syntaxe du pseudo-code utilisé pour :
   - Les opérations et leurs opérateurs
   - Exprimer des conditions
   - Exprimer des actions répétitives
3. Savoir dérouler son algorithme pour :
   - Corriger les (éventuelles) erreurs syntaxiques
   - Corriger les (éventuelles) erreurs sémantiques

Qu'est-ce qu'un pseudo-code ?

Une façon d'écrire des algorithmes dans un langage presque naturel sans référence à un langage de programmation

La syntaxe du pseudo-code :

- Ressemble parfois à celle de certains langages de programmation mais pas entièrement
- Est adaptée à la langue des usagers pour pouvoir exprimer plus facilement son raisonnement de manière formelle (pour nous ça sera en français)
- N'est pas universelle, quelques différences peuvent apparaître d'une école à une autre

À Efrei, toutes les promotions P1 vont utiliser la même syntaxe du pseudo-code

Structure générale d'un algorithme

Un algorithme se présente comme suit :

L'en-tête

La partie déclarative

Le corps de l'algorithme

```
Algorithme Nom de l'algorithme

Constantes :
          nom de la constante1 = valeur de la constante 1

Variables :
          nom de la variable1 : type variable1 (rôle)
          nom de la variable2 : type variable2 (rôle)

Début
          Action 1
          Action 2
          ….
          Action N
Fin
```

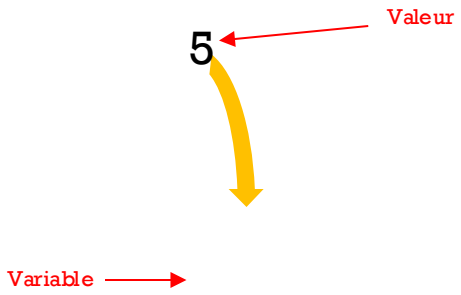
Les variables & types

efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Variables / Types

Variables

Conteneur (boîte) pour y stocker une valeur :



- Nombre : pour faire des opérations mathématiques (additions, calcul TVA, ..)
- Mot (chaîne de caractères) : pour former des phrases.

Il est possible de mettre dans un même conteneur plusieurs valeurs à des moments différents.

La nouvelle valeur à mettre écrase (annule) la valeur précédente.

Exemple:

Addition de deux nombres
- Stocker valeur 1 (conteneur 1)
- Stocker valeur 2 (conteneur 2)
- Faire la somme et … où stocker le résultat ?
  Nouvelle boîte ?

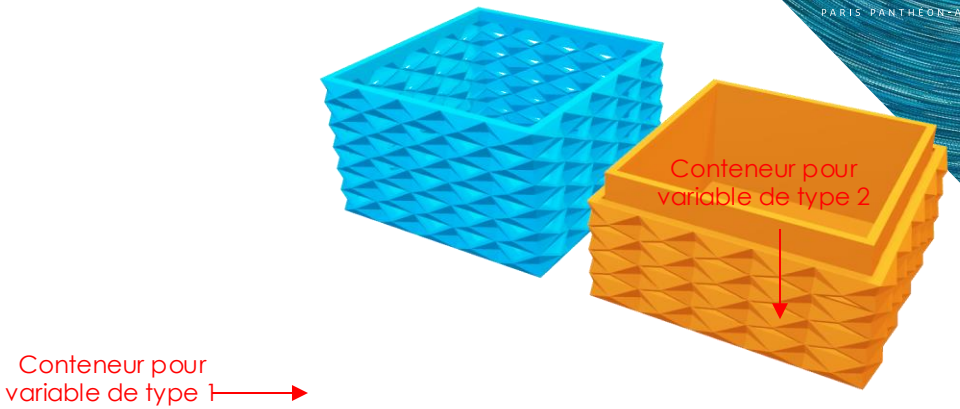
Application numérique :

- 5 + 8 = 13

Est-il possible de l'adapter pour 3.9 + 4.5 = 8.4

Variables / Types

Types



Le **type** d'une variable détermine la nature de la variable :

- Une variable d'un type donné peut prendre plusieurs valeurs de ce même type
- Il existe plusieurs types de variable

Types	Signification	Exemples de valeurs
Entier	Nombre entier	1, 3200, -450, …etc.
Réel	Nombre flottant (à virgule)	0.245, -123.65, … etc.
Caractère	Un seul caractère (une touche du clavier)	A, B, :, @, … etc.
Chaîne de caractères	Un mot, une phrase	Algorithme, hello world , …etc.
Booléen	Valeur binaire exprimant vrai ou faux	Vrai, Faux

Variables / Types

Déclaration d'une variable

1. La première chose à faire avant d'utiliser une variable est de créer une boite et lui associer un nom (appelé Label) .
2. Convention de nommage d'une variable :
   - Nom significatif : Le nom doit être en lien direct avec le rôle que prendra la variable dans le programme (Ex: nom, somme, est_vide, …)
   - Commencer par un caractère alphabétique mais :
     - Peut contenir des caractères numériques (Ex: nom1, point420, …)
     - Ne doit pas contenir des caractères spéciaux excepté le tiret du bas « _ » (Ex : nom_etudiant)
     - Ne doit pas contenir des espaces. Utiliser le tiret du bas « _ » si nécessaire
3. Associer un type à chaque variable déclarée

Variables / Types

Déclaration d'une variable

Nom_Variable : Type_variable (rôle)

Lorsque nécessaire

Exemple:

```
Algorithme Exemple_declaration_variables


Variables :
          num_etudiant : entier (identifiant)
          nom_etudiant : chaîne de caractères(nom de l'étudiant)
          note_etudiant: réel (moyenne générale)

Début
     ...
Fin
```

Les mêmes règles de nommage d'une variable sont valides pour le nom donné à un algorithme

La déclaration d'une variable permet d'avoir une place en mémoire (une boite) mais celle-ci est encore vide tant qu'aucune valeur n'est stockée dedans

Variables / Types

Constantes

1. La constante est une variable spéciale pour laquelle la valeur ne change jamais
2. Elle ne doit pas être modifiée à l'intérieur de l'algorithme
3. Par convention, les noms des constantes sont donnés en majuscule

NOM_CONSTANTE = Valeur

Exemple:

```
Algorithme Exemple_constante

Constantes :
          PI = 3.14

Variables :
          r : réel (rayon d'un cercle)
          p : réel (périmètre d'un cercle)

Début
     ...
Fin
```

Les opérateurs

Les opérateurs

Opérateur d'affectation

1. L'affectation est l'action d'assigner (attribuer) une valeur à une variable ➔ Mettre un contenu (quelque chose) dans la boîte (conteneur)
2. Le caractère qui symbolise cette opération est appelé : opérateur d'affectation
3. Dans le pseudo-code, le caractère utilisé est « ← »(la flèche) en respectant l'ordre suivant :

Nom_Variable ← Valeur ou expression

```
Algorithme Exemple_affectation

Variables :
          a, b : entier (variables diverses)

Début
      a ← 3
      b ← a
Fin
```

Lorsque plusieurs variables sont du même type, il est possible de les déclarer sur une même ligne, séparées par des virgules et mentionner le type une seule fois

La variable **a** va contenir la valeur 3
La variable **b** va contenir la valeur stockée dans **a.** Dans ce cas **b** contiendra la valeur 3

Les opérateurs

Opérateur d'affectation

Plusieurs affectations sur une même variable

Si une valeur v2 est affectée à une variable ayant déjà la valeur v1 , alors v1 sera écrasée par v2.

Exemple:

```
Algorithme Exemple_affectation_2_valeurs

Variables :
          a : entier (variable de type entier)

Début
      a ← 3
      a ← 1200
Fin
```

Après cette instruction, 'a' contient la valeur 3

À la fin de cet algorithme, **a** contiendra la valeur 1200

Incompatibilité de types

Il n'est pas possible d'affecter une valeur d'un type T1 à une variable de type T2 ≠ T1 (voir la seule exception autorisée dans les 3 slides qui suivent)

Exemple:

```
Algorithme Exemple_types_incompatibles

Variables :
          a,c : entier (variables de type entier)
          b : réel (variable de type flottant)

Début
   ✗ a ← 3.5
     c ← 76 ✓
   ✗ b ← c
Fin
```

Les opérateurs

Opérateur d'affectation

Le type caractère

- Un caractère est un type permettant de représenter des caractères numériques, caractères alphabétiques et caractères spéciaux
- Tout caractère est représenté en mémoire sous forme d'un code numérique appelé code ASCII
- Les codes numériques de tous les caractères sont rassemblés dans la table ASCII

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Les opérateurs

Opérateur d'affectation

Le codage ASCII

(American Standard Code for Information Interchange)

Ce qu'il faut retenir de la table ASCII:

- À chaque caractère est associé un code hexadécimal et son équivalent en décimal
- Les lettres majuscules ont des codes ASCII différents des lettres minuscules
- Les codes des lettres majuscules sont dans l'intervalle décimal [65, 90]
- Les codes des lettres minuscules sont dans l'intervalle décimal [97, 122]
- La différence entre une lettre majuscule d'une lettre minuscule est toujours 32

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Les opérateurs

Opérateur d'affectation

Affectation d'un caractère

- Pour qu'une valeur de type caractère ne soit pas confondue avec un nom de variable, on la met entre deux simple quotes : ' '.
- Un caractère peut être affecté à une variable de type caractère comme à une variable de type entier

```
Algorithme Exemple_affectation_caractères

Variables :
          c,a : caractères (variables de type caractère)
          b : entier (variable de type entier)
Début
      c ← 'c'
      a ← c
      b ← c
Fin
```

Le caractère 'c' est affecté à la variable c

À la fin de cet algorithme:
- la variable **a** contiendra le caractère 'c'
- La variable **b** contiendra le code ASCII du caractère 'c' , soit 99

Les opérateurs

Opérateur d'affectation

Affectation d'une chaîne de caractères

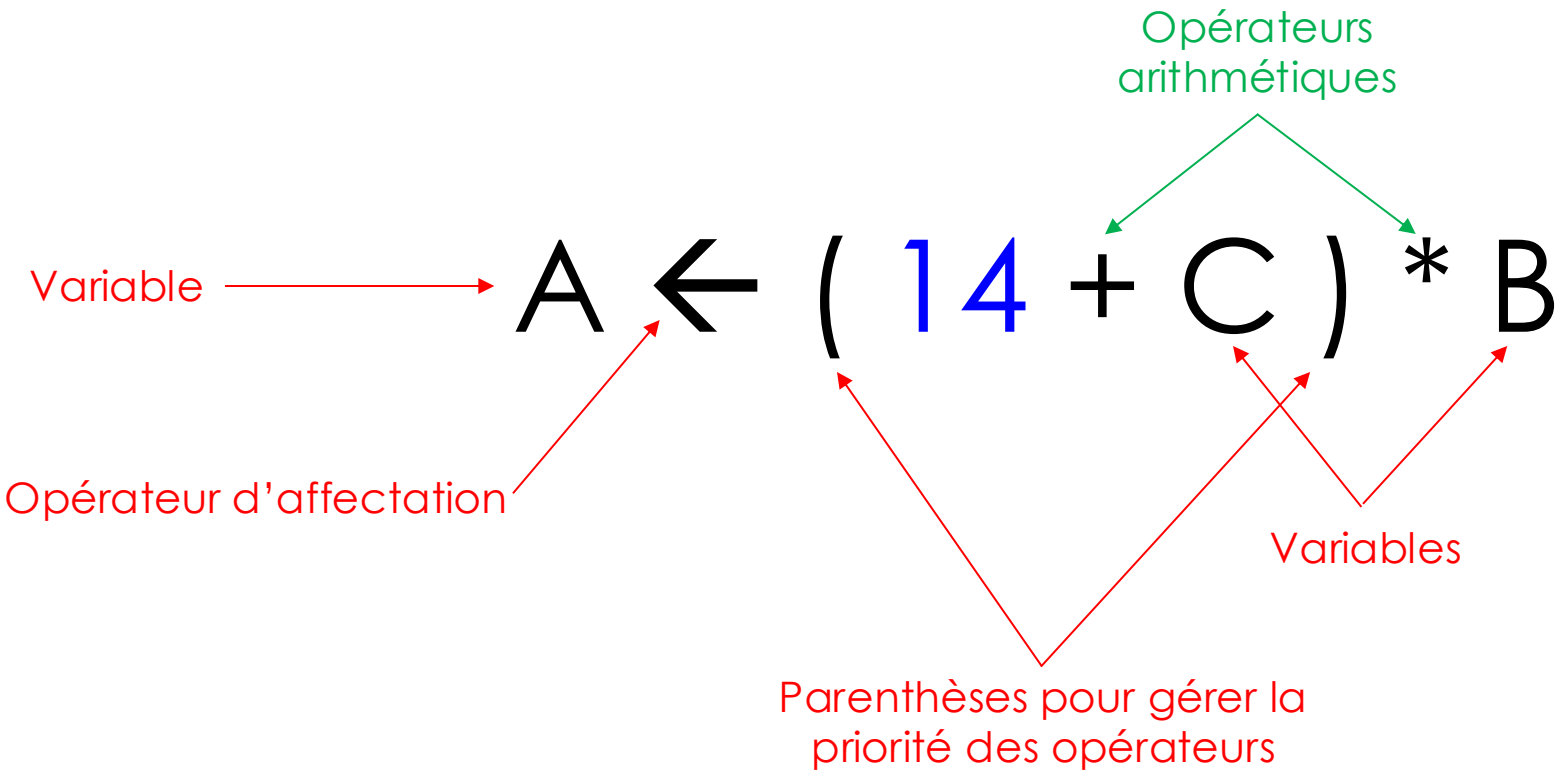
- Pour qu'une valeur de type chaîne de caractères ne soit pas confondue avec un nom de variable, on la met entre guillemets (double quote) : « »

```
Algorithme Exemple_affectation_chaine

Variables :
          str : chaîne de caractères (variables de type chaîne de caractères)

Début

      str ← « algorithmique »

Fin
```

Les opérateurs

Opérateurs arithmétiques

Formée par une combinaison d'objets numériques (entier ou réel) et des opérateurs arithmétiques



Les opérateurs

Opérateurs arithmétiques

Opérateur	Dénomination	Effet	Exemple	Résultat (avec x valant 7)
+	Opérateur d'addition	Ajoute deux valeurs	x+3	10
-	Opérateur de soustraction	Soustrait deux valeurs	x-3	4
*	Opérateur de multiplication	Multiplie deux valeurs	x*3	21
/	Opérateur de division	Divise deux valeurs (division entière)	x/3	2
%	Opérateur du modulo (uniquement pour les entiers)	Reste de la division	x%3	1
$\sqrt{\ }$	Opérateur de la racine carrée	Racine carrée d'un nombre (résultat réel)	$\sqrt{x}$	2.64

Les opérateurs

Opérateurs de comparaison

Donne un résultat booléen : Vrai ou Faux.

Opérateur de comparaison

Variable booléenne

$$A \leftarrow 3 \geq (4 * B)$$

Expressions arithmétiques

Les opérateurs

Opérateurs de comparaison

Opérateur	Dénomination	Effet	Exemple	Résultat
=	opérateur d'égalité	Compare deux valeurs et vérifie leur égalité	x = 3	Retourne Vrai si x est égal à 3, sinon Faux
<	opérateur d'infériorité stricte	Vérifie qu'une variable est strictement inférieure à une valeur	x < 3	Retourne Vrai si x est inférieur à 3, sinon Faux
≤	opérateur d'infériorité	Vérifie qu'une variable est inférieure ou égale à une valeur	x ≤ 3	Retourne Vrai si x est inférieur ou égal à 3, sinon Faux
>	opérateur de supériorité stricte	Vérifie qu'une variable est strictement supérieure à une valeur	x > 3	Retourne Vrai si x est supérieur à 3, sinon Faux
≥	opérateur de supériorité	Vérifie qu'une variable est supérieure ou égale à une valeur	x ≥ 3	Retourne Vrai si x est supérieur ou égal à 3, sinon Faux
≠	opérateur de différence	Vérifie qu'une variable est différente d'une valeur	x ≠ 3	Retourne Vrai si x est différent de 3, sinon Faux

Les opérateurs

Opérateurs logiques

Donne un résultat booléen : Vrai ou Faux.

Opérateur logique

Variable booléenne

$$A \leftarrow (B > 3) \text{ ET } (4 = B)$$

Expressions booléennes

Les opérateurs

Opérateurs logiques

Opérateur	Dénomination	Effet	Syntaxe
OU	OU logique	Vérifie qu'il y a au moins une condition réalisable	((condition1) OU (condition2))
ET	ET logique	Vérifie que toutes les conditions sont réalisées	((condition1) ET (condition2))
NON	NON logique	Inverse l'état d'une variable booléenne (retourne la valeur Vrai si la variable vaut Faux, Faux si elle vaut Vrai)	(NON condition)

Les opérateurs

Opérateurs logiques

Tables de vérité

ET		
A	B	A ET B
Faux	Faux	Faux
Faux	Vrai	Faux
Vrai	Faux	Faux
Vrai	Vrai	Vrai

OU		
A	B	A OU B
Faux	Faux	Faux
Faux	Vrai	Vrai
Vrai	Faux	Vrai
Vrai	Vrai	Vrai

NON	
A	NON A
Faux	Vrai
Vrai	Faux

Les opérateurs

Opérateur de concaténation

- La concaténation est le fait de coller bout à bout deux caractères ou deux chaînes de caractères
- L'opérateur qui permet de coller deux chaînes de caractères est le : +

Exemple :

```
Algorithme exemple_concatenation

Variables :
          c1, c2, c3, message: chaine de caractères
Début
     c1 ← « Hello »
     c2 ← «  »
     c3 ← « World ! »
     message ← c1 + c2 + c3
Fin
```

À la fin de cet algorithme, la variable message contiendra la phrase : Hello World !

Exemple

Énoncé du problème

Vous devez mesurer exactement 4 litres d'eau. Pour cela, vous disposez de :

- un récipient de 5 litres
- un deuxième de 3 litres
- un robinet

Les actions que vous pouvez utiliser pour obtenir une solution sont :

- Remplir un récipient au robinet
- Vider un récipient (à l'égout)
- Transvaser un récipient dans un autre (lorsque l'on transvase, on s'arrête lorsque le récipient source est vide ou lorsque le récipient destination est plein, et on garde le restant dans le récipient source).

Questions

1. Analysez le problème décrit. À l'aide des actions fournies, proposez une solution en utilisant le langage naturel.
2. Quelles sont les variables que l'on peut extraire de ce problème ?
3. Écrire le pseudo-algorithme correspondant à votre solution.

Exemple

Solution 1

1. Remplir le récipient 3L à partir du robinet.
2. Transvaser le récipient 3L dans le récipient 5L.
3. Remplir le récipient 3L.
4. Transvaser le récipient 3L dans le récipient 5L. (Alors, il reste 1 litre dans le récipient 3L.)
5. Vider le récipient 5L.
6. Transvaser le récipient 3L dans le récipient 5L. (Le récipient 3L est vide, le récipient 5L contient 1 L)
7. Remplir le récipient 3L.
8. Transvaser le récipient 3L dans le récipient 5L. (Nous avons alors 4 litres dans le récipient 5L.)

Exemple

Solution 2

1. Remplir le récipient 5L.
2. Transvaser le récipient 5L dans le récipient 3L. (Alors, il y a 2 litres dans le récipient 5L)
3. Vider le récipient 3L.
4. Transvaser le récipient 5L dans le récipient 3L. (le récipient 5L est vide, et le récipient 3L contient 2 litres)
5. Remplir le récipient 5L.
6. Transvaser le récipient 5L dans le récipient 3L. (un seul litre est versé dans le récipient 3L, il reste 4 litres dans le récipient 5L).

Que remarquez-vous ?

Exemple

Phase de conception

1. Quelles variables utiliser ?

Par définition, une variable est un conteneur : Dans le problème nous avons 2 conteneurs :

- Une bouteille de 5L ➔ variable **`bouteille_5L`** de type entier
- Une bouteille de 3L ➔ variable **`bouteille_3L`** de type entier

2. Comment réaliser les actions ?

- Vider : L'entier qui symbolise le vide est 0 ➔ affecter à la variable (bouteille) concernée la valeur 0
- Remplir : Affecter à la variable (bouteille) la valeur de sa capacité maximale ➔ La valeur 3 pour **`bouteille_3L`**, La valeur 5 pour **`bouteille_5L`**
- Transvaser : En fonction de la situation, associer l'opération mathématique appropriée à la variable (bouteille) concernée

Exemple

Pseudo-algorithmes

```
Algorithme solution_1

Variables :
          bouteille_5L: entier (bouteille 5L)
          bouteille_3L: entier (bouteille 3L)
Début
     bouteille_5L ← 0
     bouteille_3L ← 0
     bouteille_3L ← 3
     bouteille_5L ← bouteille_5L + bouteille_3L
     bouteille_3L ← bouteille_3L - 2
     bouteille_5L ← bouteille_5L + 2
     bouteille_5L ← 0
     bouteille_5L ← bouteille_5L + bouteille_3L
     bouteille_3L ← 3
     bouteille_5L ← bouteille_5L + bouteille_3L
Fin
```

```
Algorithme solution_2

Variables :
          bouteille_5L: entier (bouteille 5L)
          bouteille_3L: entier (bouteille 3L)
Début
     bouteille_5L ← 0
     bouteille_3L ← 0
     bouteille_5L ← 5
     bouteille_3L ← 3
     bouteille_5L ← bouteille_5L - 3
     bouteille_3L ← 0
     bouteille_3L ← bouteille_5L
     bouteille_5L ← 0
     bouteille_5L ← 5
     bouteille_5L ← bouteille_5L - 1
     bouteille_3L ← bouteille_3L + 1
Fin
```

Les entrées / sorties

Les entrées / sorties

Motivation

Considérons le pseudo-algorithme suivant qui calcule le carré de la valeur 15 :

```
Algorithme carre_quinze

Variables :
          X: entier (variable de type entier)
Début
     X ← 15 * 15
Fin
```

À la fin de cet algorithme, la variable X contiendra la valeur 225

Plusieurs questions :

Est-ce que le résultat sera visible pour l'utilisateur ? Visible à l'écran ?

Comment faire pour calculer le carré d'un autre entier ? Créer un algorithme pour chaque entier ?

Les entrées / sorties

Instruction de lecture

- Le clavier est l'entrée standard d'un ordinateur
- Pour permettre à un utilisateur de saisir une valeur au clavier, il doit utiliser la fonction « Lire » comme suit :

Lire (nom_variable)

La valeur saisie au clavier sera stockée dans la variable nom_variable

- Il n'est possible de Lire qu'une seule variable à la fois.
- La valeur saisie peut être de n'importe quel type.

```
Algorithme saisie_nombre

Variables :
           Nb : entier (Nombre saisi au clavier)
           X: entier (variable de type entier)
Début
      Lire (Nb)
      X ← Nb * Nb
Fin
```

Les entrées / sorties

Instruction d'écriture

- L'écran est la sortie standard d'un ordinateur
- Pour permettre à un utilisateur d'afficher un message à l'écran, il doit utiliser la fonction « Écrire » comme suit :

Ecrire (contenu)

- Le contenu à afficher peut être :
  - Un message : Une phrase ou un mot écrit dans le langage naturel. Il doit être mis entre guillemets « »

```
Algorithme afficher_un_message

Début
      Ecrire (« Hello World ! »)
Fin
```

L'algorithme va afficher à l'écran : Hello World !

Les entrées / sorties

Instruction d'écriture

- Pour permettre à un utilisateur d'afficher un message à l'écran, il doit utiliser la fonction « Écrire » comme suit :

Ecrire (contenu)

- Le contenu à afficher peut être :
  - Une variable : contenu d'une variable quel que soit son type. Mettre directement le nom de la variable sans les guillemets « »

```
Algorithme afficher_un_entier

Variables :
          X: entier (variable de type entier)
Début
     X ← 3
     Ecrire (X)
Fin
```

L'algorithme va afficher à l'écran : 3

```
Algorithme afficher_un_mot

Variables :
          S: chaine de caractères
Début
     S ← « Algorithmique »
     Ecrire (S)
Fin
```

L'algorithme va afficher à l'écran : Algorithmique

Les entrées / sorties

Instruction d'écriture

- Pour permettre à un utilisateur d'afficher un message à l'écran, il doit utiliser la fonction « Écrire » comme suit :

Ecrire (contenu)

- Le contenu à afficher peut être :
  - Une expression mathématique : expression utilisant des opérateurs mathématiques afin d'afficher son résultat à l'écran

```
Algorithme afficher_une_somme

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(X + Y)
Fin
```

L'algorithme va afficher à l'écran : 7

```
Algorithme afficher_un_resultat

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(X + Y * 2)
Fin
```

L'algorithme va afficher à l'écran : 11

Les entrées / sorties

Instruction d'écriture

- Pour permettre à un utilisateur d'afficher un message à l'écran, il doit utiliser la fonction « Écrire » comme suit :

Ecrire(contenu)

- Le contenu à afficher peut être :
  - Une expression booléenne : expression utilisant des opérateurs logiques afin d'afficher son résultat à l'écran

```
Algorithme afficher_vrai

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(X < Y)
Fin
```

L'algorithme va afficher à l'écran : vrai

```
Algorithme afficher_faux

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(X = Y)
Fin
```

L'algorithme va afficher à l'écran : faux

Les entrées / sorties

Instruction d'écriture

- Pour permettre à un utilisateur d'afficher un message à l'écran, il doit utiliser la fonction « Écrire » comme suit :

Ecrire(contenu)

- Le contenu à afficher peut être une phrase composée d'un message, de variables et d'expressions mathématiques et/ou booléennes séparés par des virgules ,

```
Algorithme afficher_un_resultat

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(« La produit de », X, « et », Y, « est : », X * Y)
Fin
```

L'algorithme va afficher à l'écran : Le produit de 3 et 4 est : 12

```
Algorithme afficher_une_operation

Variables :
          X, Y: entier (variable de type entier)
Début
     X ← 3
     Y ← 4
     Ecrire(X, « * », Y, « = », X * Y)
Fin
```

L'algorithme va afficher à l'écran : 3 * 4 = 12

Les entrées / sorties

Instruction d'écriture

- La fonction « Écrire » n'enclenche pas un saut de ligne automatiquement.
- Pour afficher deux contenu sur deux lignes différentes, il faut indiquer explicitement le saut de ligne en utilisant le caractère \NL (NL pour Nouvelle Ligne)

Exemples :

```
Algorithme sans_saut_de_ligne

Variables :
           X, Y: entier (variable de type entier)
Début
      X ← 3
      Y ← 4
      Ecrire(« X = », X)
      Ecrire(« Y = », Y)
Fin
```

L'algorithme va afficher à l'écran :
X = 3Y = 4

```
Algorithme avec_saut_de_ligne

Variables :
           X, Y: entier (variable de type entier)
Début
      X ← 3
      Y ← 4
      Ecrire(« X = », X, «\NL»)
      Ecrire(« Y = », Y)
Fin
```

L'algorithme va afficher à l'écran :
X = 3
Y = 4

Les entrées / sorties

Application

Quelle est la sortie de l'algorithme suivant ?

```
Algorithme exercice

Variables :
           a : entier (variable de type entier)
           x : réel
           c : caractère
Début
      a ← -7
      x ← 1.5
      c ← '?'
      Ecrire(a , «\NL»)
      Ecrire(«a+c \NL»)
      Ecrire(x , «\NL»)
      Ecrire(«a*x \NL»)
      Ecrire(x * a, «\NL»)
Fin
```

Les structures de contrôle : instruction de condition

efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Les structures de contrôle

L'instruction conditionnelle : « Si »

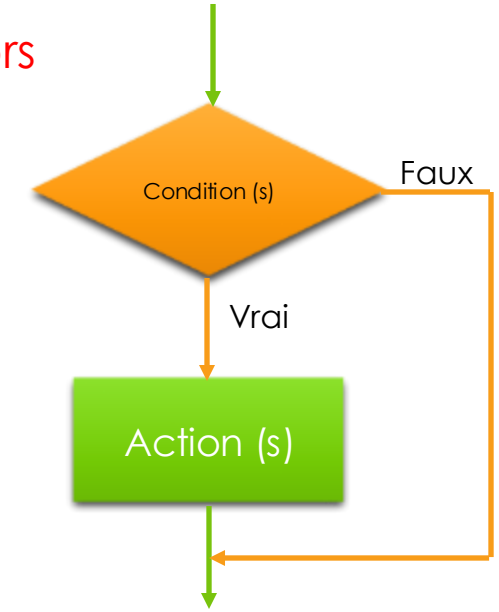
- Utilisée si une (ou plusieurs) action (s) doit (doivent) s'exécuter sous réserve qu'une condition soit vérifiée.
- La condition peut être une simple expression booléenne, ou une composition de plusieurs expressions booléennes

Syntaxe :

Si (condition) alors

action (s)

Fin Si



Exemple :

```
Algorithme valeur_absolue

Variables :
            X: entier (variable de type entier)
Début
      Lire(X)
      Si (X < 0) alors
            X ← -X
      Fin Si
Fin
```

Les structures de contrôle

L'instruction alternative : « Si - Sinon »

- Utilisée lorsqu'il y a un bloc d'instructions à exécuter uniquement si la condition est vérifiée et un autre bloc d'instructions à exécuter uniquement lorsque la condition n'est pas vérifiée.

Syntaxe :

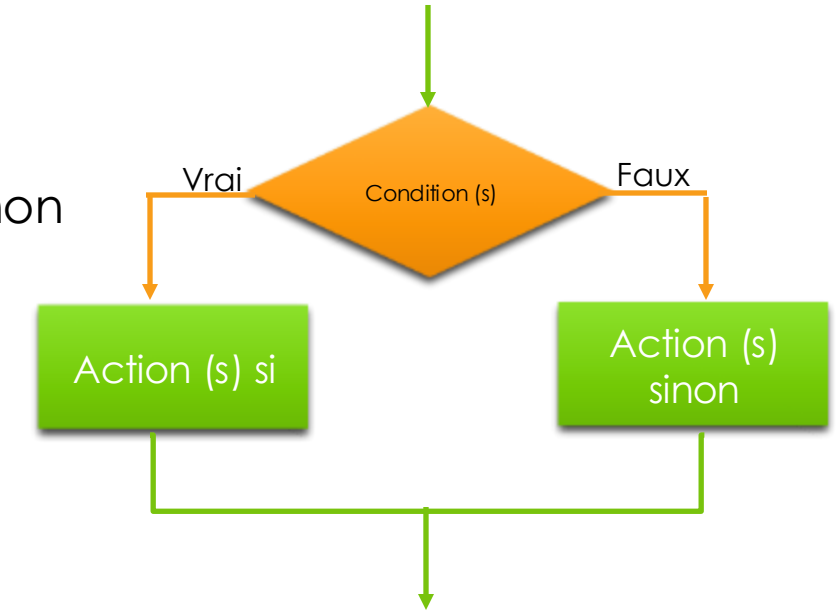
Si (condition) alors

    action (s) si

Sinon

    action (s) sinon

Fin Si



Exemple :

```
Algorithme positif_negatif

Variables :
            X: entier (variable de type entier)
Début
      Lire(X)

      Si (X < 0) alors
            Ecrire(X, « est négatif \NL »)
      Sinon
            Ecrire(X, « est positif ou nul »)
      Fin Si
Fin
```

Les structures de contrôle

Les instructions de test imbriquées

- Utilisée lorsqu'il y a plusieurs cas à tester.

Syntaxe :

Si (condition 1) alors

action (s) si 1

Sinon

Si (condition 2) alors

action (s) si 2

Sinon

…

Fin Si

Fin Si

Exemple :

```
Algorithme positif_negatif_nul

Variables :
          X: entier (variable de type entier)
Début
     Lire(X)

     Si (X < 0) alors
          Ecrire(X, « est négatif \NL »)
     Sinon
          Si (X > 0) alors
               Ecrire(X, « est positif \NL»)
          Sinon
               Ecrire(X, « est nul \NL»)
          Fin Si
     Fin Si
Fin
```

Application

Écrire un algorithme qui :

1. Demande à l'utilisateur de Lire 3 entiers : a, b, c représentants les coefficients d'une équation du second degré
2. Calcule la valeur de delta
3. Affiche le nombre de solutions possibles en fonction du signe de delta

Les structures de contrôle

L'instruction de test imbriquées : « Selon – que »

```
Algorithme jours_semaine

Variables :
          jour: entier (numéro du jour)
Début
     Ecrire(« Lire un entier : »)
     Lire(jour)

     Si (jour = 1) alors
          Ecrire(« Lundi \NL »)
     Sinon
          Si (jour = 2) alors
               Ecrire(« Mardi \NL»)
          Sinon
               Si (jour = 3) alors
                    Ecrire(« Mercredi \NL»)
               Sinon
                    ….
               Fin Si
          Fin Si
     Fin Si
Fin
```

Trop long

Difficile à lire / comprendre

Utiliser l'instruction « Selon – que »

Les structures de contrôle

L'instruction de test imbriquées : « Selon – que »

Syntaxe :

Selon (variable)

cas valeur 1 : action 1

cas valeur 2 : action 2

…

cas valeur N : action N

Sinon : action par défaut

Fin Selon

Exemple :

```
Algorithme jours_semaine_selon_que

Variables :
            jour: entier (numéro du jour)
Début
      Ecrire(« Lire un entier : »)
      Lire(jour)

      Selon (jour)
            cas 1 : Ecrire(« Lundi \NL »)
            cas 2 : Ecrire(« Mardi \NL »)
            cas 3 : Ecrire(« Mercredi \NL »)
            cas 4 : Ecrire(« Jeudi \NL »)
            cas 5 : Ecrire(« Vendredi \NL »)
            cas 6 : Ecrire(« Samedi \NL »)
            cas 7 : Ecrire(« Dimanche \NL »)
            Sinon : Ecrire(« Entier invalide ! »)

      Fin Selon
Fin
```

Les structures de contrôle: les boucles

efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Les boucles

Intérêt

- Utilisées lorsqu'un bloc d'instructions se répète plusieurs fois
- Au lieu d'écrire les mêmes instructions plusieurs fois ➔ écrire le bloc une seule fois à l'intérieur d'une boucle.
- Les tours (répétitions) réalisés par la boucle sont appelés: itérations
- Trois types de boucles
  - Pour
  - Tant que
  - Répéter – Tant que

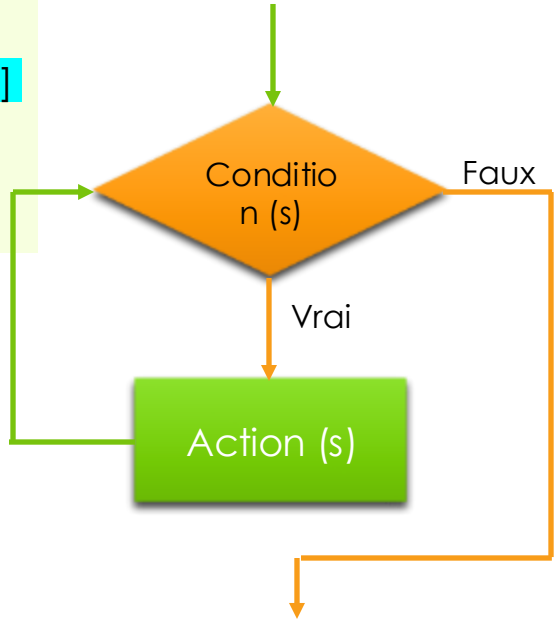
Les boucles

La boucle « Pour »

- Cette boucle est utilisée lorsqu'on connaît exactement le nombre d'itérations à effectuer.
- Elle manipule une variable « compteur d'itérations » prenant ses valeurs dans l'intervalle [début, fin]

Fonctionnement :

- La variable compteur est initialisée à la valeur « début » et l'algorithme exécute le bloc d'instructions.
- À la prochaine itération, la variable compteur est incrémentée de 1 automatiquement si [Pas] non indiqué, sinon la valeur de [Pas] lui est ajoutée.
- L'algorithme s'arrête lorsque la variable compteur dépasse la valeur « fin »



Syntaxe :

Pour variable_compteur ← début À fin [Pas] Faire

action (s)

Fin Pour

Les boucles

La boucle « Pour »

Exemple :

```
Algorithme somme_cent_premiers_entiers_pour

Variables :
            sum: entier (somme )
            cpt: entier (compteur d'itérations)
Début
      sum ← 0

      Pour cpt ← 1 À 100 Faire
            sum ← sum + cpt
      Fin Pour

      Ecrire(« La somme des 100 premiers entiers est : », sum)
Fin
```

Les boucles

La boucle « Tant que»

- Cette boucle est utilisée lorsque l'on ne connaît pas le nombre d'itérations à effectuer.
- Elle permet de tester si une condition donnée sous forme d'une expression booléenne est vérifiée ou pas.

Syntaxe :

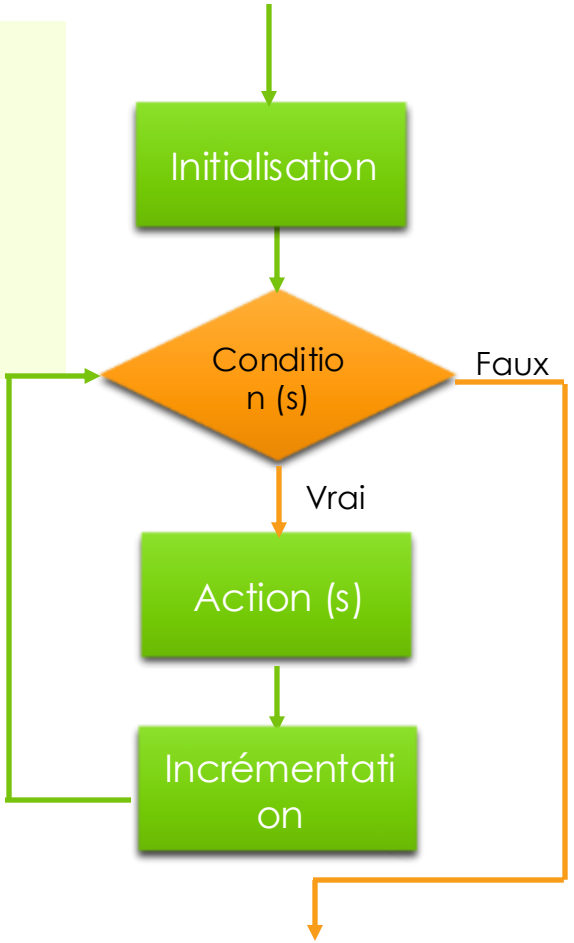
Initialisation

Tant que (expression logique vérifiée) Faire

    action (s)

    Incrémentation

Fin Tant que



Les boucles

La boucle « Tant que»

```
Algorithme somme_cent_premiers_entiers_tantque

Variables :
            sum: entier (somme )
            cpt: entier (compteur d'itérations)
Début
      sum ← 0
      cpt ← 1
      Tant que cpt ≤ 100 Faire
            sum ← sum + cpt
            cpt ← cpt + 1
      Fin Tant que

      Ecrire(« La somme des 100 premiers entiers est : », sum)
Fin
```

Les boucles

La boucle « Répéter - Tant que»

- Cette boucle est utilisée lorsque l'on ne connaît pas le nombre d'itérations à effectuer.
- Elle permet de tester si une condition donnée sous forme d'une expression booléenne est vérifiée ou pas.
- Utilisée lorsqu'on souhaite effectuer la première itération sans passer par la condition

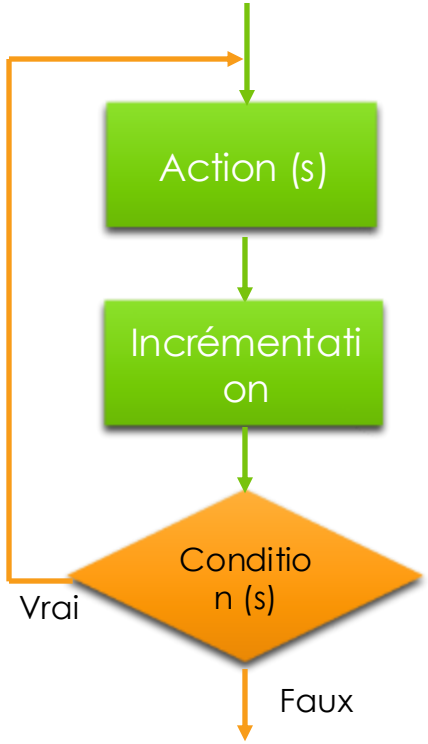
Syntaxe :

Répéter

action (s)

Incrémentation

Tant que (expression logique vérifiée)



Les boucles

La boucle « Répéter - Tant que»

```
Algorithme saisie_operateur

Variables :
           op: entier (opérateur mathématique )

Début

      Répéter
            Ecrire(« Choisir une opération  \NL»)
            Ecrire(« Entrer 1 pour l'addition \NL»)
            Ecrire(« Entrer 2 pour la soustraction \NL»)
            Ecrire(« Entrer 3 pour la multiplication \NL»)
            Ecrire(« Entrer 4 pour la division \NL»)
            Lire(op)
      Tant que (op < 1 OU op > 4)

Fin
```

Bonnes pratiques

Les bonnes pratiques

Les commentaires

- Ce sont des portions de texte (écrites dans un langage naturel) ajoutées à l'algorithme dans le but d'apporter une explication supplémentaire à une instruction ou à un bloc d'instructions.
- En algorithmique, pour reconnaître un commentaire, nous choisissons de l'écrire entre les deux caractères : (* et *)
- Même si les commentaires font partie de l'algorithme, ils sont néanmoins ignorés à son exécution

```
Algorithme produit_100_entiers

Variables :
           i: entier (compteur d'itérations)
           mul : entier (résultat de la multiplication)

Début
     mul ← 1 (* initialisé à 1 car c'est l'élément neutre en multiplication *)
     Pour i ← 1 À 100 faire
           mul ← mul * i
     Fin pour
     Ecrire(« Le résultat de la multiplication est : », mul)
Fin
```

Les bonnes pratiques

L'indentation

- C'est le retrait de lignes introduit à un bloc d'instructions pour montrer à quelle structure de tests ou à quelle boucle il appartient.
- L'indentation est nécessaire pour la lisibilité d'un algorithme.
- Sur le clavier, l'indentation est obtenue par la touche « tabulation »

```
Algorithme positif_negatif_nul

Variables :
            X: entier (variable de type entier)
Début
    Lire(X)

    Si (X < 0) alors
        Ecrire(X, « est négatif \NL »)
    Sinon
        Si (X > 0) alors
            Ecrire(X, « est positif \NL»)
        Sinon
            Ecrire(X, « est nul \NL»)
        Fin Si
    Fin Si
Fin
```

Les bonnes pratiques

Être interactif avec l'utilisateur

- Avant la saisie de toute valeur, il est nécessaire d'afficher à l'écran un message indiquant clairement à l'utilisateur ce qu'il doit Lire, l'intervalle de valeurs autorisées, …etc.

- L'affichage d'un résultat doit être accompagné d'un message clair et bien formaté.

Exemple

```
Algorithme date_de_naissance

Variables :
          jour, mois, annee: entier
Début
     Ecrire(« Lire votre jour de naissance : »)
     Lire(jour)
     Ecrire(«\NLLire votre mois de naissance : »)
     Lire(mois)
     Ecrire(«\NLLire votre année de naissance : »)
     Lire(annee)

     Ecrire(« \NLVous êtes né(e) le : », jour, « / », mois, « / », annee)

Fin
```

Les bonnes pratiques

Éviter les variables inutiles

- Ne jamais déclarer de variables ni de constantes si elles ne sont pas manipulées dans l'algorithme.

Exemple

```
Algorithme saisie_operateur

Constantes :
            PI = 3.14

Variables :
            a, b, i: entier
            sum : entier
Début

      sum ← 0
      Pour i ← 1 À 20 Faire
            sum ← sum + i
      Fin Pour

Fin
```